

Blind Spot Detection with Active Intervention

Smart Vehicular Systems 2024-2025

Pablo Sebastian Vargas Grateron - 0001137787
{pablo.vargasgrateron@studio.unibo.it}

July 25, 2025

Indice

1	Introduzione	2
2	Requisiti	3
2.1	Requisiti funzionali	3
2.2	Requisiti non funzionali	3
3	Design	5
3.1	General Block Diagram del funzionamento del sistema . .	5
3.2	Sensori e posizionamento	5
3.2.1	Radar: specifiche e posizionamento	5
3.2.2	Telecamera: specifiche e montaggio	6
3.3	Pipeline di elaborazione dei dati	7
3.3.1	Radar: clustering e filtraggio dei target	7
3.3.2	Telecamera: rilevamento dei veicoli con YOLOv5 .	9
3.3.3	Fusione sensoriale: integrazione dei dati per la deci- sione	10
3.4	Pubblicazione degli eventi su MQTT	10
3.4.1	Configurazione	10
3.4.2	Pubblicazione degli eventi	10
3.5	Controllo del veicolo e gestione degli interventi	11
4	Sviluppo e deployment	12
4.1	Prerequisiti	12
4.2	Librerie e tecnologie utilizzate	12
4.3	Esecuzione del codice	13
5	Conclusioni	14
5.1	Problemi e limitazioni riscontrati nel progetto	14
5.2	Potenzialità e punti di forza del sistema	14
5.3	Sviluppi futuri e possibili miglioramenti	15

Introduzione

L'obiettivo di questo progetto è sviluppare un sistema ADAS per il Blind Spot Detection, in grado di rilevare veicoli presenti nei punti ciechi di un'automobile. Il sistema è progettato per ridurre il rischio di incidenti, avvertendo il conducente della presenza di potenziali minacce durante le manovre di cambio corsia e, in situazioni di emergenza, intervenendo autonomamente per garantire la sicurezza.

Requisiti

2.1 Requisiti funzionali

I requisiti funzionali del progetto sono i seguenti:

- **Tempo reale:** progettare e implementare un sistema in grado di operare in tempo reale, garantendo una risposta rapida alle situazioni di pericolo rilevate nei punti ciechi del veicolo.
- **Simulazione e validazione:** sviluppare il sistema utilizzando il simulatore CARLA, al fine di testare e validare il funzionamento in scenari controllati e realistici.
- **Rilevamento accurato:** identificare in maniera precisa eventuali veicoli o ostacoli presenti nei punti ciechi, minimizzando falsi positivi e falsi negativi.
- **Allerta e intervento:** implementare meccanismi efficaci per avvertire il conducente in caso di potenziali rischi e, quando necessario, intervenire autonomamente per prevenire incidenti.
- **Comunicazione degli eventi con MQTT:** gestire la trasmissione degli eventi rilevati tramite protocollo MQTT.
- **Compatibilità con periferiche di guida:** garantire che il sistema sia utilizzabile con volante e pedali.

2.2 Requisiti non funzionali

Oltre ai requisiti funzionali, il sistema deve soddisfare diversi requisiti non funzionali, che definiscono le caratteristiche qualitative e le restrizioni progettuali:

- **Prestazioni e tempo reale:** il sistema deve elaborare i dati dei sensori e generare allarmi o interventi con latenza minima, garantendo risposte rapide durante le manovre del veicolo.
- **Affidabilità:** il sistema deve fornire rilevazioni accurate e consistenti, minimizzando falsi positivi e falsi negativi anche in condizioni ambientali variabili (pioggia, nebbia, luce solare intensa).

- **Costo contenuto:** il progetto deve utilizzare sensori relativamente economici, come radar e telecamere, evitando soluzioni costose come lidar ad alta precisione, per garantire la fattibilità commerciale.
- **Scalabilità e modularità:** il sistema deve essere facilmente adattabile a diversi modelli di veicoli e facilmente aggiornabile con nuovi algoritmi di elaborazione dati..

Design

In questo capitolo viene illustrato il processo di progettazione del sistema di rilevamento dei veicoli nei punti ciechi, descrivendo nel dettaglio le scelte architettoniche, i modelli sensoriali adottati, l'organizzazione dei moduli software e le logiche decisionali che regolano il funzionamento del Blind Spot Detection System (BSD).

3.1 General Block Diagram del funzionamento del sistema

L'architettura del sistema è organizzata come una pipeline composta da cinque fasi principali, ciascuna responsabile di una parte specifica del processo di rilevamento dei veicoli nei punti ciechi.

La Figura 3.1 illustra la struttura generale del sistema e il flusso completo delle informazioni attraverso i moduli *Environment*, *Sensors*, *Perception*, *Analysis* e *Actuation*.

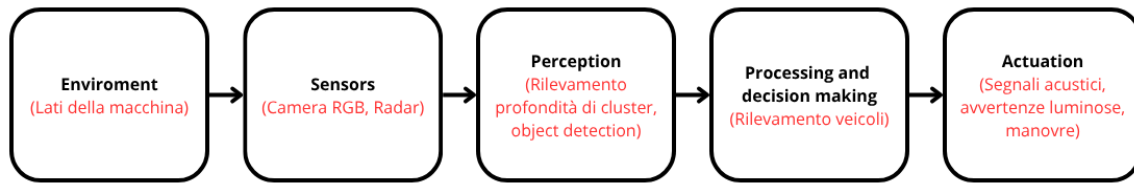


Figure 3.1: Architettura generale del sistema di Blind Spot Detection.

3.2 Sensori e posizionamento

In questa sezione vengono descritti i sensori utilizzati per il sistema di Blind Spot Detection e il loro posizionamento sul veicolo. L'obiettivo è illustrare le scelte progettuali legate ai dispositivi impiegati, alla loro configurazione e alle aree di copertura garantite, elementi fondamentali per ottenere un rilevamento affidabile dei veicoli nei punti ciechi.

3.2.1 Radar: specifiche e posizionamento

Il sensore responsabile del rilevamento della presenza e della distanza dei veicoli nei punti ciechi è il radar. Nel progetto sono stati utilizzati due radar, posizionati simmetricamente sugli specchietti laterali del veicolo e orientati verso la parte posteriore. Questa configurazione consente di coprire in modo efficace entrambe le zone laterali e di monitorare l'avvicinamento di veicoli provenienti da dietro.

Ogni radar presenta le seguenti caratteristiche operative:

- **Campo visivo orizzontale (FOV):** 40°
- **Campo visivo verticale:** 10°
- **Raggio massimo di rilevamento:** 10 metri

La scelta del posizionamento e dell'orientazione è stata effettuata in modo da garantire la copertura completa delle aree tipicamente associate ai punti ciechi, includendo sia la zona immediatamente adiacente al veicolo sia una porzione più arretrata della carreggiata, così da rilevare tempestivamente veicoli che si stanno avvicinando a velocità più elevate.

La Figura 3.2 mostra il posizionamento dei due radar rispetto alla geometria del veicolo e le relative zone di copertura.

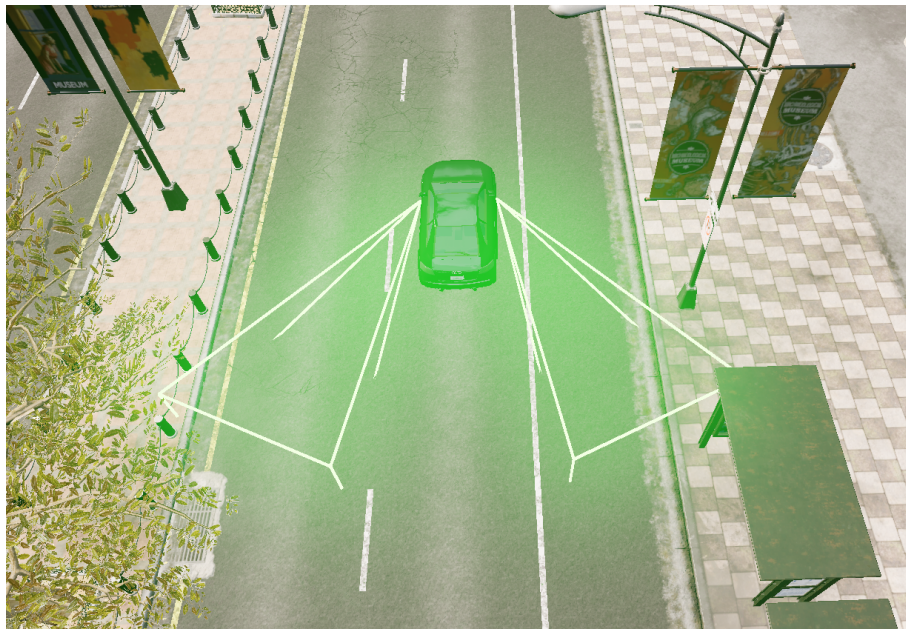


Figure 3.2: Schema del posizionamento dei radar sui lati del veicolo e dei relativi campi visivi.

3.2.2 Telecamera: specifiche e montaggio

Analogamente ai radar, il sistema utilizza due telecamere posizionate sui lati del veicolo, con un Field of View leggermente più ampio rispetto ai radar stessi. Questa configurazione consente di osservare una porzione maggiore dei punti ciechi, ampliando l'area utile al riconoscimento visivo dei veicoli. L'ampio campo di osservazione delle telecamere risulta particolarmente vantaggioso nelle fasi successive della pipeline, dove la visione artificiale permette di identificare con precisione la presenza di veicoli nelle zone laterali.

Le telecamere adottate presentano le seguenti specifiche:

- **Risoluzione dell'immagine:** 800×600 pixel
- **Tipologia di immagine:** colore (RGB)
- **Frequenza di acquisizione:** 10 FPS

Queste caratteristiche risultano sufficienti per fornire un flusso visivo stabile e informativo, mantenendo al contempo un carico computazionale compatibile con un sistema in tempo reale.

La Figura 3.3 mostra un esempio delle due telecamere laterali e le rispettive aree di copertura.



(a) Telecamera sinistra



(b) Telecamera destra

Figure 3.3: Posizionamento delle due telecamere laterali e relative aree di copertura.

3.3 Pipeline di elaborazione dei dati

Di seguito vengono illustrate le due pipeline implementate e il processo di fusione tra di esse.

3.3.1 Radar: clustering e filtraggio dei target

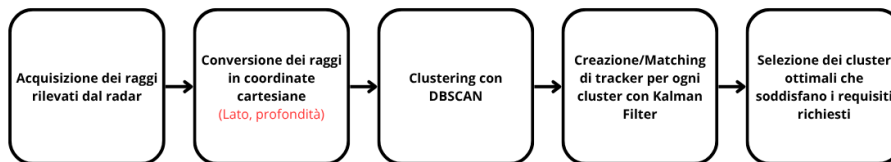


Figure 3.4: Pipeline del sistema radar.

Il processo di elaborazione dei dati del radar si articola in cinque fasi principali, come mostrato in Figura 3.4.

La conversione dei raggi rilevati dal sensore in coordinate cartesiane, rappresentate da coppie *lato–profondità*, è stata scelta per poter applicare

l'algoritmo di clustering **DBSCAN** (1), con l'obiettivo di raggruppare i raggi vicini tra loro in base alla distanza. Se i raggi ricevuti condividono una distanza simile dal centroide, essi vengono assegnati allo stesso cluster, indicando la presenza di un oggetto significativo nel raggio d'azione del radar.

La conversione dai dati del radar, espressi in termini di profondità r , azimuth θ e elevazione ϕ , alle coordinate cartesiane (x, y) avviene secondo le seguenti formule:

$$\begin{cases} x = r \cdot \cos(\phi) \cdot \cos(\theta) \\ y = r \cdot \cos(\phi) \cdot \sin(\theta) \end{cases} \quad (3.1)$$

I cluster generati dall'algoritmo **DBSCAN** vengono successivamente elaborati tramite un *filtro di Kalman* (2) bidimensionale, che sfrutta le proprie proprietà di predizione e aggiornamento per stimare lo stato futuro degli oggetti rilevati e ridurre il rumore dovuto a misurazioni imprecise. In questo modo, ogni cluster può essere tracciato in maniera più stabile nel tempo, migliorando l'affidabilità del sistema radar nella rilevazione e nel monitoraggio dei target.

Lo stato del tracker è rappresentato dal vettore

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix},$$

dove (x, y) sono le coordinate del cluster e (v_x, v_y) le velocità associate.

Predizione La predizione dello stato al passo successivo avviene tramite la matrice di transizione **F**:

$$\mathbf{x}_{k|k-1} = \mathbf{F}\mathbf{x}_{k-1|k-1}, \quad \mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^\top + \mathbf{Q}, \quad (3.2)$$

dove **P** è la matrice di covarianza e **Q** il rumore di processo.

Aggiornamento Quando viene ricevuta una nuova misura $\mathbf{z}_k$, il filtro aggiorna lo stato con:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}\mathbf{x}_{k|k-1}, \quad (3.3)$$

$$\mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}, \quad (3.4)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^\top\mathbf{S}_k^{-1}, \quad (3.5)$$

$$\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k\mathbf{y}_k, \quad (3.6)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1}, \quad (3.7)$$

dove $\mathbf{R}$ rappresenta la matrice di covarianza del rumore di misura e $\mathbf{H}$ la matrice di osservazione (che misura solo posizione).

Finalmente, dopo aver filtrato i cluster tramite i rispettivi *tracker di Kalman*, vengono ottenuti soltanto i cluster che mantengono una persistenza temporale, scartando così eventuali falsi positivi generati da rumori esterni. Successivamente, tutti i cluster rilevati e filtrati vengono inviati alla pipeline di fusione dei dati.

3.3.2 Telecamera: rilevamento dei veicoli con YOLOv5

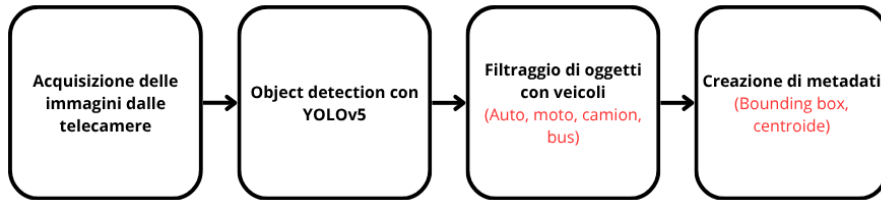


Figure 3.5: Pipeline del sistema telecamera.

Il processo di elaborazione dei dati della telecamera si articola in cinque fasi principali, come mostrato in Figura 3.5.

Dopo l'acquisizione dell'ultimo frame generato dalla telecamera, il frame viene fornito in input al modello **YOLOv5** (3) per il rilevamento degli oggetti presenti nella scena. L'utilizzo di YOLO permette di ottenere le *bounding box*, ossia le coordinate che racchiudono ciascun oggetto rilevato, insieme alla relativa tipologia, con tempi di elaborazione molto rapidi: la media calcolata è di circa 110 ms per frame.

Una volta ottenute le *bounding box* di ciascun oggetto presente nella scena, si procede al filtraggio dei rilevamenti per selezionare esclusivamente le classi di interesse: auto, camion, bus e moto. I dati relativi agli oggetti filtrati vengono quindi inviati alla pipeline di fusione dei dati.

3.3.3 Fusione sensoriale: integrazione dei dati per la decisione

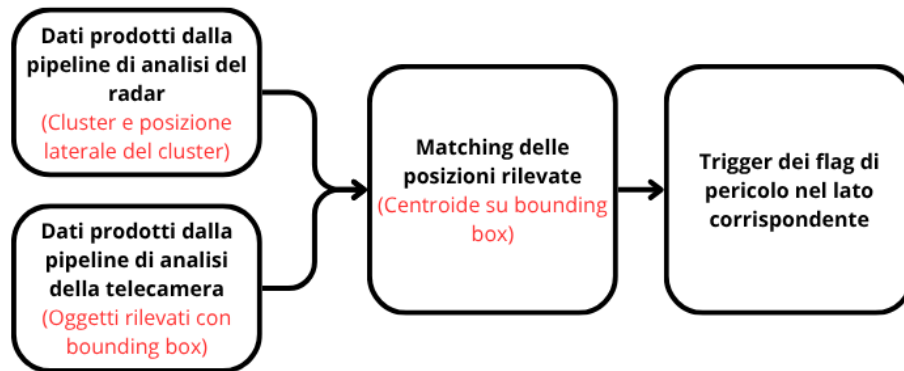


Figure 3.6: Pipeline del sistema in cui avviene la fusione dei dati provenienti dai sensori.

Questa rappresenta l'ultima fase della pipeline generale di elaborazione dei dati, illustrata in Figura 3.6.

La pipeline di fusione sensoriale si occupa di eseguire entrambe le pipeline in modo sincronizzato e, una volta ottenuti i risultati di ciascuna, calcola e confronta le posizioni dei centroidi generati dal radar con le *bounding box* prodotte dalla telecamera. Se viene individuata una corrispondenza che soddisfa i requisiti stabiliti, la presenza del veicolo nel punto cieco dell'auto viene confermata. In caso di esito positivo della fusione, vengono attivati i flag di sinistra o di destra per segnalare il pericolo.

3.4 Pubblicazione degli eventi su MQTT

In questa sezione viene descritta la configurazione del client MQTT e il processo di pubblicazione degli eventi generati dal sistema.

3.4.1 Configurazione

Per la pubblicazione degli eventi su MQTT, è necessario configurare il client con i parametri del broker. Nel nostro caso, il broker utilizzato è `test.mosquitto.org` sulla porta 1883, e il topic di pubblicazione è definito come `svs_blind_spot_detection`.

Nota Per scopi dimostrativi è stato utilizzato il broker pubblico `mosquitto.org`; tuttavia, per applicazioni reali, si consiglia di impiegare broker con autenticazione e connessioni sicure.

3.4.2 Pubblicazione degli eventi

La pubblicazione degli eventi avviene in due situazioni principali:

- **[WARNING] - Vehicle detected on the LEFT/RIGHT side:** questo messaggio viene inviato quando l'utente manifesta l'intenzione di cambiare corsia e nel punto cieco è presente un veicolo.
- **[INTERVENTION] - Vehicle detected on the LEFT/RIGHT side:** questo messaggio viene inviato quando l'utente esegue una manovra diretta verso un pericolo presente nel punto cieco.

3.5 Controllo del veicolo e gestione degli interventi

Parallelamente al processamento dei dati, viene eseguito un controllo periodico sulle azioni dell'utente e sui flag di pericolo. In particolare, l'utente deve manifestare l'intenzione di cambiare corsia utilizzando le frecce direzionali e il veicolo deve trovarsi a una velocità superiore a 30 km/h; in questo modo, il sistema di *Blind Spot Detection* verifica se esiste un pericolo nella direzione desiderata.

Nel caso in cui il flag della luce direzionale di un lato sia attivo e il flag di pericolo corrispondente a quel lato risulti anch'esso vero, il sistema entra in una prima fase di avvertimento, in cui l'utente viene segnalato tramite un allarme e sullo schermo appare l'indicazione di un pericolo imminente nella direzione selezionata.

Se, nonostante le avvertenze della prima fase, il conducente tenta comunque di svoltare verso la direzione pericolosa, il sistema interviene applicando una leggera resistenza al volante. Tale resistenza è calibrata in modo da non compromettere il controllo del veicolo, ma serve come ultima linea di avvertimento, fornendo all'utente un feedback fisico sulla presenza di un pericolo.

Sviluppo e deployment

4.1 Prerequisiti

Per eseguire correttamente questo progetto, sono richieste alcune specifiche minime di hardware:

- Almeno 16 GB di RAM.
- Una scheda grafica con almeno 8 GB di VRAM.
- Un sistema operativo a 64 bit; è preferibile un processore con più di 8 core.
- Un controller esterno collegato al computer. Il progetto è progettato per l'utilizzo con i controlli del simulatore, ma è possibile impiegare un controller generico da console modificando il file `wheel_config.ini`.

Oltre alla repository del progetto disponibile all'indirizzo <https://github.com/Zettaiki/bsd-svs-2025>, è necessario avere installato il simulatore **CARLA 0.9.15** e un ambiente **Python 3.7.x**.

4.2 Librerie e tecnologie utilizzate

Le principali librerie utilizzate nel progetto, oltre a quelle native di Python, sono le seguenti:

- **carla**: per l'interfacciamento con il simulatore CARLA.
- **NumPy**: per la gestione di dati, immagini, matrici e operazioni di calcolo lineare.
- **paho-mqtt**: per la comunicazione tramite client MQTT.
- **ultralytics**: per l'utilizzo del modello *YOLOv5 nano*.
- **scikit-learn**: per la clusterizzazione dei punti radar tramite l'algoritmo DBSCAN.
- **manual_control_steeringwheel**: codice incluso negli esempi del simulatore CARLA.

4.3 Esecuzione del codice

Per l'esecuzione del progetto, si può fare riferimento alla repository GitHub disponibile all'indirizzo <https://github.com/Zettaiki/bsd-svs-2025>, dove il file `README.md` contiene le istruzioni dettagliate per avviare il codice del sistema ADAS per il *Blind Spot Detection (BSD)*.

Conclusioni

5.1 Problemi e limitazioni riscontrati nel progetto

Il progetto sviluppato si è dimostrato generalmente robusto ed è stato testato in diverse situazioni, tra cui curve, tratti rettilinei e scenari complessi. Nella maggior parte dei casi non si riscontrano problemi significativi; tuttavia, il sistema presenta alcune limitazioni.

In particolare, è necessario calibrare attentamente il *sampling* dei sensori e definire con precisione le condizioni operative in cui l'ADAS deve intervenire. Il sistema può infatti generare falsi positivi quando il veicolo si trova in movimento in presenza di altri veicoli parcheggiati in fila. In scenari urbani a bassa velocità, questo non rappresenta un problema critico, ma costituisce comunque una debolezza del sistema.

Un'ulteriore limitazione riguarda l'impiego del modello di intelligenza artificiale e delle telecamere. Nonostante YOLOv5 sia un modello robusto e caratterizzato da elevate capacità di generalizzazione in diversi scenari, il suo funzionamento dipende comunque dalla qualità delle immagini acquisite. Situazioni come l'occlusione parziale o totale della telecamera, condizioni ambientali sfavorevoli (pioggia intensa, scarsa illuminazione, controllo) o la presenza di ostacoli che impediscono una chiara visualizzazione dell'area monitorata possono ridurre drasticamente l'accuratezza del rilevamento, aumentando il rischio di mancata individuazione dei veicoli presenti nei punti ciechi.

Va inoltre sottolineato che una delle principali debolezze del sistema deriva dalla sua struttura a doppio sensore. In particolare, nel caso in cui uno dei due sensori dovesse guastarsi o fornire dati errati, l'ADAS potrebbe non essere in grado di rilevare i veicoli presenti nella scena.

5.2 Potenzialità e punti di forza del sistema

Grazie alla pipeline a doppio sensore, uno dei principali punti di forza di questo sistema è la riduzione dei falsi positivi in scenari ad alta velocità. Il lavoro ibrido di rilevamento della profondità e conferma del veicolo, unito a diversi filtri e analisi, consente al sistema di distinguere efficacemente quando un elemento rappresenta effettivamente un veicolo e, allo stesso tempo, se tale elemento costituisce un rischio per il veicolo. Questo è possibile grazie alla capacità del sistema di valutare sia la distanza sia la

classe dell'oggetto presente nel punto cieco.

Oltre a quanto già descritto, il codice è stato ottimizzato in modo tale da poter eseguire tutti i calcoli necessari in tempo reale, con un ritardo massimo di 200 ms tra la sincronizzazione dei frame e l'elaborazione dei dati relativi ai pericoli presenti nei punti ciechi.

5.3 Sviluppi futuri e possibili miglioramenti

Tra i possibili miglioramenti del sistema vi è l'implementazione di algoritmi più avanzati per il tracciamento dei veicoli rilevati dal radar, in grado di stimare con maggiore precisione la velocità e la traiettoria degli oggetti. Questo consentirebbe di ridurre ulteriormente i falsi positivi, ad esempio quelli generati da veicoli parcheggiati o da elementi statici presenti nell'ambiente circostante.

Inoltre, l'integrazione di ulteriori sensori o l'adozione di tecniche di fusione sensoriale più sofisticate potrebbe migliorare la robustezza del sistema in scenari complessi, come il traffico urbano denso o le condizioni meteo avverse. Infine, ottimizzazioni sul piano computazionale potrebbero permettere una riduzione del ritardo nella pipeline di elaborazione, aumentando così l'affidabilità del sistema nella guida reale.

Bibliography

- [1] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *KDD*, pp. 226–231, ACM, 1996.
- [2] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.